



Quand Power BI passe à l'échelle: transformer un rapport en plateforme BI

Merci à nos mécènes

Ils rendent l'événement possible.



Quand Power BI passe à l'échelle: transformer un rapport en plateforme BI



Carole CAZENAVETTE

Consultante data freelance

3C Data

Bio

Consultante data freelance avec 15 ans d'expérience, j'accompagne mes clients de la modélisation et la définition des KPIs métier jusqu'à la mise en place de plateformes data complètes.

J'interviens sur l'ensemble de la chaîne : architecture data, pipelines ETL, modèle sémantique et reporting, sur les stacks Microsoft BI / Azure / Databricks.

Carole.cazenavette@3cdata.fr - [Carole Cazenavette](#) | [LinkedIn](#)

Quand Power BI passe à l'échelle: transformer un rapport en plateforme BI



Tom VINANT

Consultant data

Bio

Consultant data avec 6 ans d'expérience, acquise dans des domaines métiers variés.
Expertise BI et modélisation de données, principalement autour des solutions Microsoft.

J'accompagne nos clients dans leurs projets, de la rédaction des spécifications à la livraison de rapports qui transforment leurs données en insights actionnables.

tom.vinant.pro@gmail.com - [Tom Vinant](#) | [LinkedIn](#)

01

PRESENTATION DU CONTEXTE



Le projet en un coup d'œil

Domaine



Reporting officiel de KPIs de forecast

Émissions & réductions CO₂ / Méthane

Multi-usage :

Reporting standardisé + exploration + benchmark

Sources :

Bases de données, CSV, fichiers Excel



100+

pages de rapport



3+

types de sources



<10min

fréquence de refresh cible



Multi

rapports & usages

2022

POC

2023–2025

Rapport Power BI en exploitation

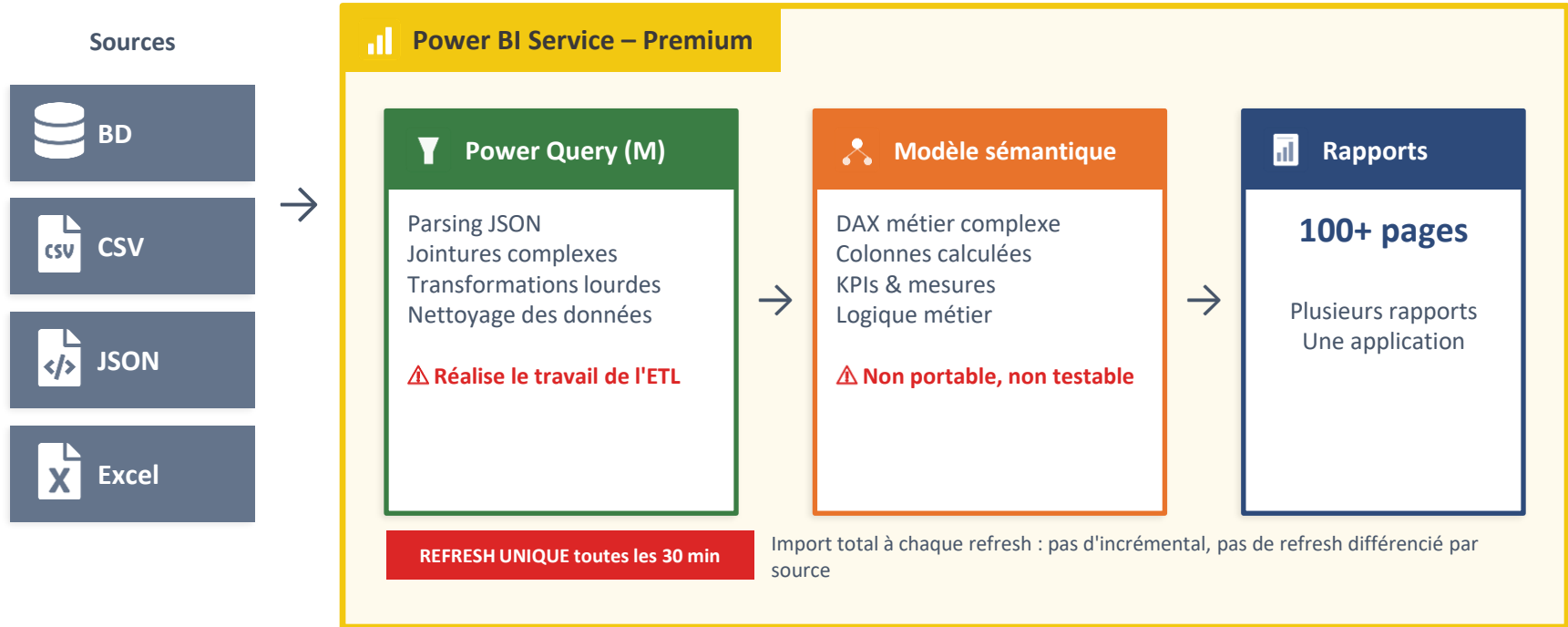
2025

Migration plateforme data

2026

Exploitation

Architecture initiale: « classique » Power BI



Tout est dans Power BI : ingestion, transformation, logique métier, rendu.

Tout commence par un rapport « simple »

1 Un besoin

2022

Un rapport Power BI pour suivre les KPIs de forecast CO₂/CH₄

Quelques sources, un périmètre maîtrisé

Import classique, refresh programmé

→ Ça fonctionne.



2 La croissance

2023 – 2024

100+ pages, plusieurs rapports

Nouveaux usages : exploration, benchmark, export dans Excel

Demandes métier croissantes

→ Ça tient, mais ça devient plus complexe à maintenir.



3 Le point de rupture

2025

Code M et DAX devenu complexes

Refresh trop lent vs. besoin métier

Maintenance fragile, pas de versioning

→ Le rapport ne scale plus.



**À quel moment vous savez que vous avez
dépassé les limites du reporting ?**

Les signaux d'alerte



Robustesse

Modèle fragile, erreurs fréquentes, ruptures en chaîne



Logique enfermée

Code M complexe (parsing JSON, jointures) et DAX éparpillé dans les mesures, non testable, non réutilisable



Fréquence d'import

Fréquence de refresh trop faible par rapport au besoin métier

Différencier les fréquences en fonction des sources



Multiples usages

Exploration, benchmark, export... un seul rapport ne peut pas tout servir



Portabilité

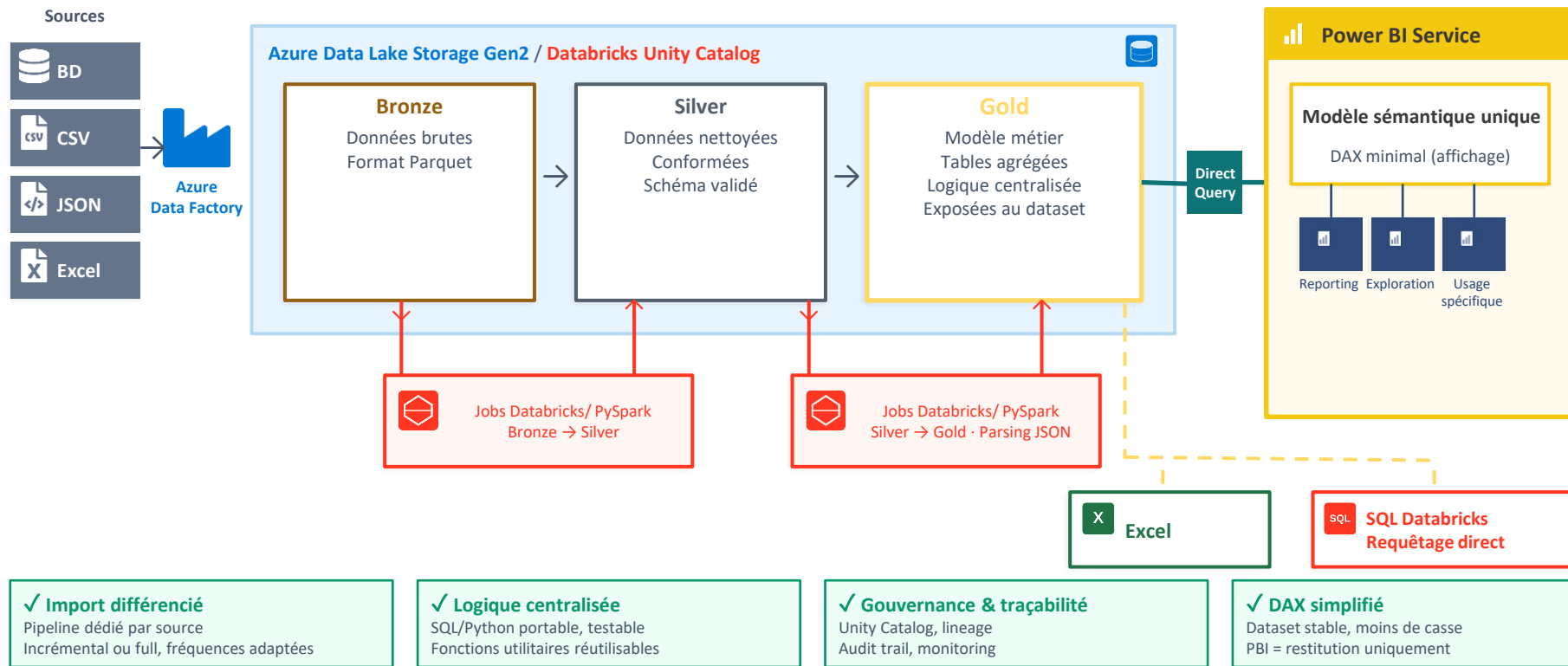
Code M/DAX non portable : impossible de réutiliser en Python, SQL ou dans un autre outil. Power BI réalise le travail de l'ETL.



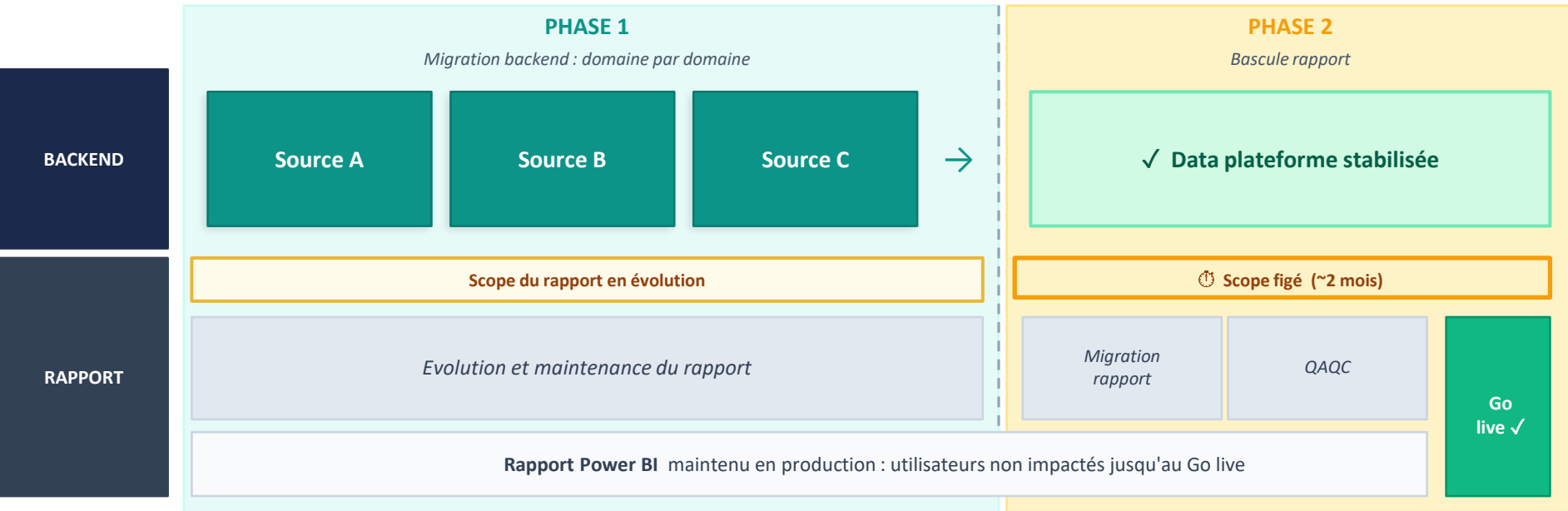
Traçabilité

Aucun versioning du code source, pas de lineage, pas d'audit trail

Architecture cible



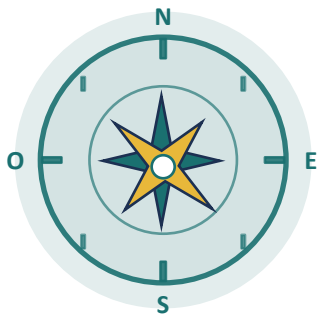
Organisation du projet



02

FOCUS SUR DES POINTS STRUCTURANTS

Focus sur 3 points structurants



01 Logique métier en amont

Où fait-on vivre les règles métier ? Gold layer vs modèle sémantique

02 Utilisation du Direct Query

Import vs DirectQuery : quelle fréquence de fraîcheur est nécessaire ?

03 Bascule des rapports Power BI

Comment migrer sans repartir de zéro et capitaliser sur l'existant ?

2.1

TRANSFORMATIONS ET REGLES METIER DANS LA COUCHE GOLD

FOCUS 1: Règles métier dans la couche Gold - Le contexte

Avant : les données et règles métier étaient enfermées dans le modèle sémantique Power BI.



Refresh non maîtrisé

Toutes les sources liées à un seul cycle de refresh, impossible de les gérer indépendamment



Catalogue & data quality

Légère documentation des indicateurs, aucun contrôle qualité sur les données produites



Code non portable

Logique en DAX / Power Query (M)



Mutualisation limitée

Données bloquées dans PBI, pas de datahub transverse ni de réutilisation inter-outils

FOCUS 1: Règles métier dans la couche Gold - Les décisions



Toute la logique métier dans la couche Gold

La couche Gold est versionnée, exposable, interrogeable par n'importe quel outil en SQL.



PySpark pour les transformations complexes

Parsing JSON, enrichissements, logiques conditionnelles lourdes.



Refresh décorrélés par domaine de données

Sources hétérogènes : une source critique à 10 min ne doit pas attendre une source plus lente à 1h.

Le découplage permet aussi le chargement incrémental domaine par domaine.



Gold exposée en SQL, ouverte à l'extérieur

L'objectif long terme est d'enrichir le datahub entreprise, avec une approche produit.

La couche Gold est structurée pour être consommée par d'autres outils (reporting, exploration, data quality, requêtage en langage naturel...)

FOCUS 1: Règles métier dans la couche Gold - Bilan

	Avant	Après
Logique métier	Éparpillée entre DAX, Power Query et non versionnée	Centralisée dans la couche Gold, versionnée dans Git, testable, documentée
Qualité des données	Non contrôlée : erreurs découvertes par les métiers, pas à la source	Contrôles intégrés dans le pipeline, alertes monitorées sous Databricks
Fréquence de refresh	Dataset unique : toutes les sources au même rythme, impossible à adapter	Décorrélé par domaine : 10 min / 1h / à la demande selon les besoins + loading incrémental
Réutilisabilité	Connexion via le modèle sémantique Power BI	SQL exposé via le datahub, requêtage en langage naturel Accès à la donnée plus scalable dans notre cas du Direct Query



Compromis

Les KPIs dépendants du contexte de sélection (ex: ratio etc..) restent dans le modèle sémantique PBI:

→ Deux endroits pour les KPIs : Gold pour les absolus, PBI pour les calculs contextuels.



Avantage non prévu

Les métadonnées structurées dans la couche Gold permettent aux métiers de requêter en langage naturel sans passer par un rapport Power BI.

Si c'était à refaire : même décision. Documenter dès le départ la frontière Gold / PBI pour les KPIs.

2.2

UTILISATION DU MODE DIRECT QUERY

FOCUS 2: Direct query - Le besoin et le contexte

Avant : Chargement en mode import toutes les 30 min, une seule fréquence pour toutes les sources.



Augmenter la fréquence de refresh

Une source de données nécessite un refresh toutes les 10 min



Contrainte infra

Pas de tenant Fabric disponible, pas de licence Power BI Premium.
Environnement Databricks + Azure Data Lake via Unity Catalog.



Gestion des refreshs par source

Plusieurs sources avec des fréquences très différentes.

FOCUS 2: Direct query - Comment ça fonctionne?

À chaque interaction utilisateur dans le rapport



× N visuels sur la page = temps de chargement total perçu par l'utilisateur

Conséquences concrètes

Latence perçue

5-15 sec par changement de visuel selon la complexité de la page

Concurrence

5 utilisateurs simultanés = 5× les requêtes sur le même cluster.

Coût

Cluster actif = facturation. Même pour un simple clic de filtre.

FOCUS 2: Direct query - Retour terrain

Ce que DirectQuery promet

Pas de refresh long

Données « temps réel »

Requêtes générées auto (query folding)

Apparente simplicité



Conséquences

Latence à chaque interaction utilisateur

Requêtes envoyées avec **un max de 10 requêtes simultanées**, le reste est mis en queue

Nombreuses requêtes SQL générées pour le moindre visuel

Modélisation plus exigeante : cardinalité, tables d'agrégats, mode dual pour les dimensions etc..

FOCUS 2: Direct query - Décisions et optimisations



Cluster Databricks

Sizing adapté à la charge réelle + autosutdown rapide → facturation uniquement à l'usage



Tables d'agrégat

Pré-calculs dans la couche Gold → réduction du volume de requêtes envoyées au cluster



Rapport allégé

Moins de visuels par page, moins d'interaction entre les visuels, bouton Apply all → moins de requêtes



Mode Dual pour les dimensions

Dimensions chargées en Import → zéro requête cluster pour les filtres statiques



À retenir : les optimisations réduisent le coût et améliorent les performances, mais ne suppriment pas la latence perçue. Le choix DirectQuery implique un compromis assumé sur l'expérience utilisateur.

FOCUS 2: Direct query - Conclusion

✓ Ce qu'on a obtenu

- ✓ Données fraîches toutes les 10 min, SLA métier tenu
- ✓ Modèle de coût à la demande

⚠ Les compromis

- ⚠ **Latence** : Le gain relative à la fraîcheur des données **s'accompagne d'une latence d'utilisation des rapports.**
- ⚠ **Coût difficile à prévoir**: Pics de consommation liés à la charge et à la complexité des rapports. Nécessite un monitoring actif.
- ⚠ **Modélisation pointue**: Toute erreur (cardinalité, agrégats mal dimensionnés) se paie immédiatement en latence et en coût compute.

✓ **DirectQuery : bonne décision seulement dans ce contexte**: refresh 10 min requis, pas de capacité Fabric disponible



Si c'était à refaire :

- Clarifier et quantifier de façon plus concrète avec le métier les conséquences du refresh très fréquent
- Tester un modèle composite avec des données chaudes (direct query) et froides (import)

2.3

Bascule des rapports existants

Focus 3: Bascule des rapports PBI - Besoin et contexte

Contexte : Plusieurs rapports (thin reports) soit 100+ pages, connectés à un unique modèle sémantique en live connection.



Périmètre: un modèle sémantique et des rapports

Modèle sémantique : tables renommées, colonnes refactorisées, mesures DAX à faire évoluer
Rapports: tous les visuels, filtres, signets et interactions sont à reconstruire



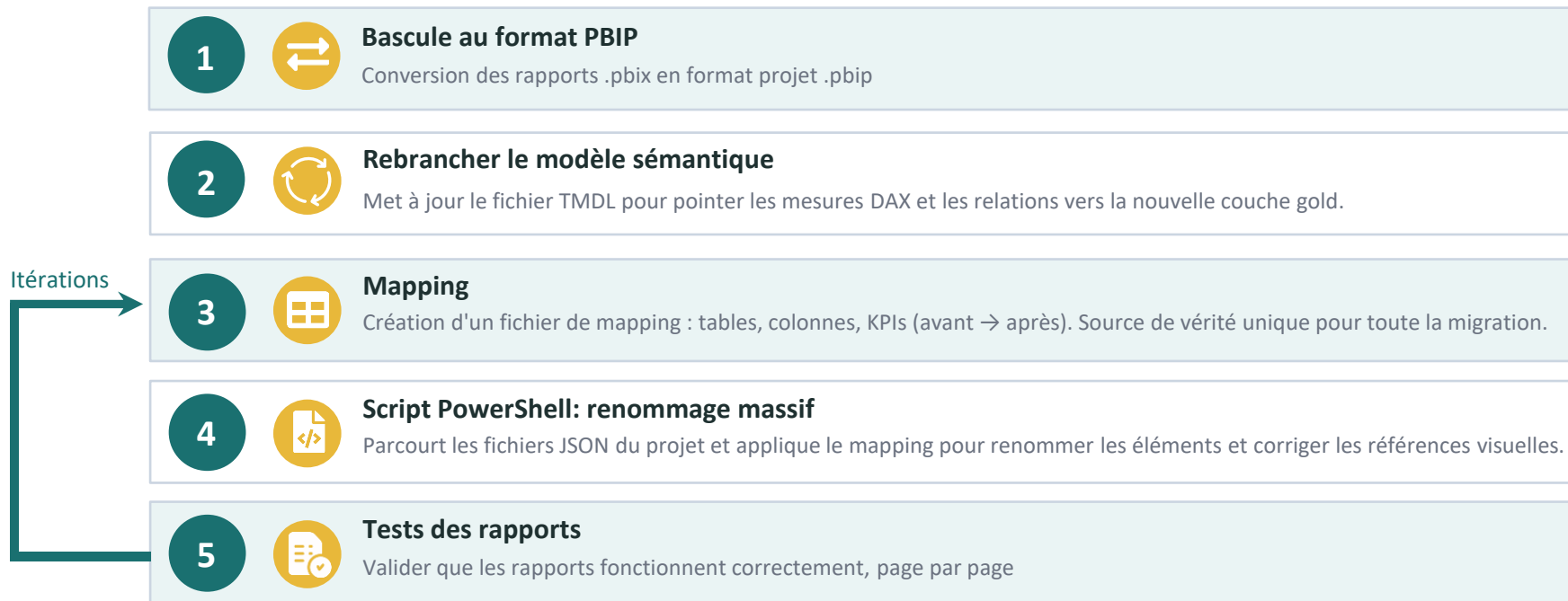
Risque de régression important

Pour chaque élément à reconstruire manuellement, il y a un risque d'erreur humaine



Contrainte de temps

Focus 3: Bascule des rapports PBI - Méthode



Focus 3: Bascule des rapports PBI - Bilan

✓ Migration automatisée : réécriture minimale, processus reproductible, risque contenu

✓ Ce qu'on a obtenu

✓ Migration rapide, peu de modification manuelle

✓ Bascule robuste avec un risque contenu

⚠ Les compromis

⚠ Vigilance lors du renommage massif car nébuleux

⚠ Construire le mapping → chronophage

03

Et Fabric dans tout ça?

Ce qu'on aurait fait dans tous les cas

Trois décisions d'architecture prises en amont, valables quelle que soit la plateforme

01

Architecture en couches

02

Migration sans coupure de service

03

Séparation du modèle et des rapports

Ces choix précèdent le choix de la plateforme. Ils sont aussi ce qui rend l'architecture portable.

Avantages du stack actuel vs apports réels de Fabric

Stack actuel (Databricks + ADF + ADLS + Power BI)

Païement à l'usage

Les clusters Databricks ne tournent que quand ils travaillent. Aucune capacité réservée à vide.

Maturité Databricks

Puissant pour les transformations complexes à grande échelle. Gestion fine du compute des clusters.

Flexibilité par brique

Chaque service évolue indépendamment. Pas de dépendance à un écosystème unique.

Fabric

DirectLake

Mirroring dans OneLake

Copie des données des sources avec la fonctionnalité de mirroring.

DirectLake

Performances sans SQL endpoint

Lecture directe des fichiers Delta depuis OneLake. Performances proches de l'Import, fraîcheur en temps réel.

Deployment Pipelines

CI/CD natif

XMLA write inclus dans la capacité Fabric. Plus besoin de publish manuel.

Gouvernance

Linéage et accès centralisés

Purview intégré sur l'ensemble du pipeline. Un seul périmètre de gouvernance au lieu de trois.

MERCI !

Des questions ?

✉ carole.cazenavette@3cdata.fr • ✉ tom.vinant.pro@gmail.com

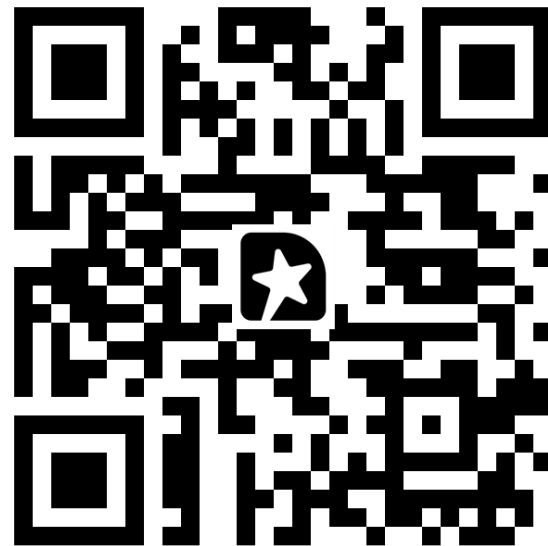


Votre avis compte.

Cette session était la vôtre.
Aidez-nous à construire la prochaine.

2 minutes suffisent pour nous dire ce qui vous a marqué,
ce que vous souhaitez approfondir, et ce que nous pouvons améliorer.
Chaque retour est lu et pris en compte.

Merci de votre participation 🙏



Quand Power BI passe à l'échelle : transformer un rapport en plateforme BI